



باسمه تعالی
Distributed optimization and learning
 گزارش پروژه دوم



شماره دانشجویی: 810101521

نام و نام خانوادگی: محمدعرفان مفید

ایمیل: m.erfan.mofid@ut.ac.ir

Review of Distributed penalty dual optimization for our problem:

General Distributed Penalty Primal-Dual Algorithm

Consider a network of N agents (nodes) aiming to solve a constrained optimization problem. The global problem is formulated as:

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_{i=1}^N f_i(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{g}(\mathbf{x}) \leq \mathbf{0}, \\ & \mathbf{x} \in \mathcal{X}, \end{aligned}$$

to solve this in a distributed manner, we employ the **Penalty Primal-Dual Algorithm**. We introduce a **Penalty-Lagrangian Function** $H(x, \mu)$ defined as:

$$H(x, \mu) = \sum_{i=1}^N f_i(x) + N \cdot \mu^T [g(x)]^+$$

Information Fusion (Consensus) Before updating, agent i gathers information from its neighbors N_i to approximate the global state. It computes weighted averages

$$v_{x,i}^{(t)} = \sum_{j \in N_i} a_{ij} x_j^{(t)}, \quad v_{\mu,i}^{(t)} = \sum_{j \in N_i} a_{ij} \mu_j^{(t)}$$

Primal Update (Descent) Agent i updates its primal variables x_i to minimize the Penalty-Lagrangian H . It takes a x step in the direction of the negative subgradient of H with respect to

$$x_i^{(t+1)} = P_X \left(v_{x,i}^{(t)} - \alpha_t \nabla_x H_i(x, v_{\mu,i}^{(t)}) \right)_{x=v_{x,i}^{(t)}}$$

Phase 3: Dual Update (Ascent)

Agent i updates the dual prices μ_i to penalize violations more heavily. This is a gradient ascent step on the dual variables:

$$\mu_i^{(t+1)} = \left[v_{\mu,i}^{(t)} + \alpha_t \cdot g(v_{x,i}^{(t)}) \right]^+$$

If the constraint is violated ($g > 0$), the price μ increases, forcing the primal update in the next step to work harder to satisfy it.

If the constraint is satisfied ($g \leq 0$), the price decreases (but stays non-negative).

Beamforming:

We apply this framework to the Convex-Concave Procedure (CCP) approximation of the beamforming problem. This time each ccp loop is solved by **Distributed Penalty Primal-Dual Algorithm**:

Problem Variables

Primal Variable (W): The beamforming matrix of size $N \times M$. Each node n controls the n -th row.

Dual Variable (μ): A vector containing prices for all constraints.

Indices $1, \dots, K$: SINR constraints for each user.

Indices $K + 1, \dots, K + N$: Per-antenna power constraints.

$$H(W, \mu) = \sum_{n=1}^N \sum_{m=1}^M |w_{n,m}|^2 + N \sum_{k=1}^K \mu_k [g_k^{\text{SINR}}(W)]^+ + N \sum_{n=1}^N \mu_{K+n} [g_n^{\text{POW}}(W)]^+$$

Where:

$$g_k^{\text{SINR}}(W) = \gamma_k \left(\sum_{j \neq k} |h_k^H w_j|^2 + \sigma_k^2 \right) - \left(2\Re \left\{ \left(w_k^{(t)} \right)^H h_k h_k^H w_k \right\} - |h_k^H w_k^{(t)}|^2 \right)$$

$$g_n^{\text{POW}}(W) = \sum_{m=1}^M |w_{n,m}|^2 - P_n \leq 0$$

Algorithm:

In this setup, **Agent n** is responsible for optimizing the **n-th row** of the matrix, denoted as $r_n \in \mathbb{C}^{1 \times M}$. However, it maintains a local estimate of the full matrix W (denoted) and full dual vector μ to calculate gradients.

A. Information Fusion (Consensus)

Before updating, Agent n averages its estimates with its neighbors N_n to ensure the network agrees on the global state (interference and prices).

$$W_{\text{cl}} \leftarrow \sum_{j \in N_n \cup \{n\}} c_{nj} W_{\text{cl},j}$$

$$\mu \leftarrow \sum_{j \in N_n \cup \{n\}} c_{nj} \mu_j$$

For our first simulation the graph is considered to be fully connected. (Weight = (1/N) * ones(N,N);)

B. Primal Update (Descent)

Agent n updates its row r_n to minimize the Lagrangian. The update rule is:

$$r_n^{(k+1)} = r_n^{(k)} - \alpha_k \cdot \nabla_{r_n} L$$

The gradient $\nabla_{r_n} L$ is composed of three parts:

1. Objective Gradient (Power Minimization):

$$\nabla_{r_n} |W|_F^2 = 2r_n$$

2. SINR Penalty Gradient:

For each user u, if the constraint is violated (or active), the dual price μ_u activates a penalty. The gradient contribution depends on whether the beam j belongs to user u (Signal) or not (Interference).

Signal Component (Beam u):

The linearized signal term is linear in w_u . The gradient of with respect to the n-th element of w_u is:

$$\nabla_{w_{nu}} L \leftarrow -\mu_u \cdot 2 \cdot h_{nu} \cdot \left(h_u^H w_u^{(t)} \right)^*$$

$$-\mu_u S_u^{\text{lin}}$$

Interference Component (Beam $j \neq u$):

The interference term is quadratic. The gradient of $\mu_u \gamma_u |h_u^H w_j|^2$ with respect to the n-th element of w_j is:

$$\nabla_{w_{nj}} L \leftarrow \mu_u \cdot \gamma_u \cdot 2 \cdot h_{nu} \cdot (h_u^H w_j)^*$$

3. Power Constraint Penalty:

For the local power constraint $\sum_m |w_{nm}|^2 \leq P_n$ associated with dual variable μ_{K+n} :

$$\nabla_{r_n} L \leftarrow \mu_{K+n} \cdot 2r_n$$

C. Dual Update (Ascent)

Agent n updates the prices to penalize violations.

$$\mu^{(k+1)} = \max\left(0, \mu^{(k)} + \alpha_k \cdot g(W_{\text{ld}})\right)$$

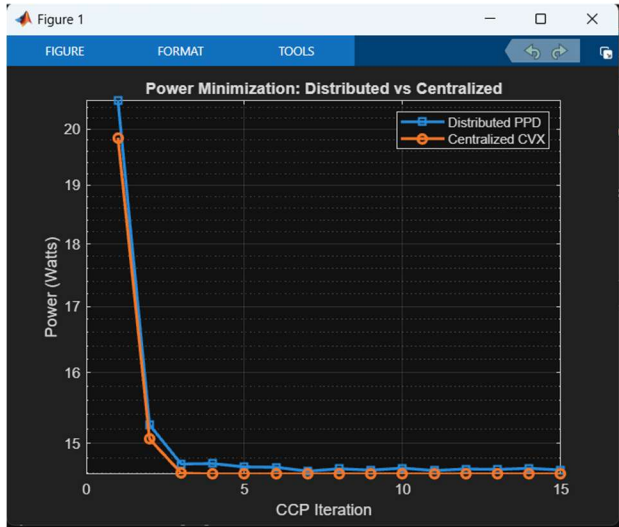
Where $g(W)$ is the vector of constraint values:

- $\text{SINR}_u: \gamma_u(\text{Interference}) - \text{Signal}_{\text{lin}}$
- $\text{Power}_n: |r_n|^2 - P_n$

Result:

For step size due to fact that with each ccp update we are getting closer to minimum therefore the step size for each ccp loop is considered to be:

$$\alpha_k = 10/(1 + k) * ccp$$



```
Total PPD Inner Iterations: 225000

=====
FINAL RESULT VERIFICATION
=====

--- Power Constraint Check ---
Antenna 1: Power = 1.7467 W, Limit = 10.0000 W [OK]
Antenna 2: Power = 3.7149 W, Limit = 10.0000 W [OK]
Antenna 3: Power = 6.3590 W, Limit = 10.0000 W [OK]
Antenna 4: Power = 2.8161 W, Limit = 10.0000 W [OK]
>> Result: All Power Constraints SATISFIED.

--- SINR Constraint Check ---
User 1: Target = 10.00, Achieved = 10.0136 [OK]
User 2: Target = 10.00, Achieved = 10.0299 [OK]
User 3: Target = 10.00, Achieved = 10.0439 [OK]
>> Result: All SINR Constraints SATISFIED.
```

For comparison each ccp loop is solved each time by ppd and cvx and as the results are demonstrating in the picture's

The result's are equal.

:The effect of different Graph's

Topologies:

Scenario 1: Fully Connected (Doubly Stochastic)

- **Graph:** Every node is connected to every other node.
- **Topology:** A Complete Graph (K_4).
- **Matrix:** All weights are equal ($1/N = 0.25$).

Scenario 2: Ring Connected (Stochastic)

- **Graph:** Nodes form a closed loop (1–2–3–4–1).
- **Topology:** A Cycle Graph. Each node connects only to its left and right neighbors.
- **Matrix:** Weights are distributed among neighbors (e.g., $1/3$ for self, left, right).

Scenario 3: Line Connected (Stochastic)

- **Graph:** Nodes form a daisy chain (1–2–3–4).
- **Topology:** A Path Graph. Endpoints (1 & 4) have only 1 neighbor; inner nodes (2 & 3) have 2.
- **Matrix:** Row stochastic. Endpoints put $1/2$ weight on their single neighbor.

2. Time-Varying (Periodic) Topologies

In these scenarios, the graph is **never** fully connected at any single instant. It switches between two "Disconnected" states. Convergence relies on the memory of the agents bridging the gap over time.

- **State 1:** Vertical pairs only (1 connects to 3; 2 connects to 4).
- **State 2:** Horizontal pairs only (1 connects to 2; 3 connects to 4).

Scenario 4: Periodic (Switch every 1 iter)

- **Graph:** Switches state every single iteration (Fast Switching).

Scenario 5: Periodic (Switch every 50 iter)

- **Graph:** The network stays in State 1 for 50 iterations, then State 2 for 50 iterations.

Scenario 6: Periodic (Switch every 100 iter)

- **Graph:** Slower switching (Period = 200 total).

Scenario 9: Periodic (Switch every 500 iter)

- **Graph:** Very slow switching.

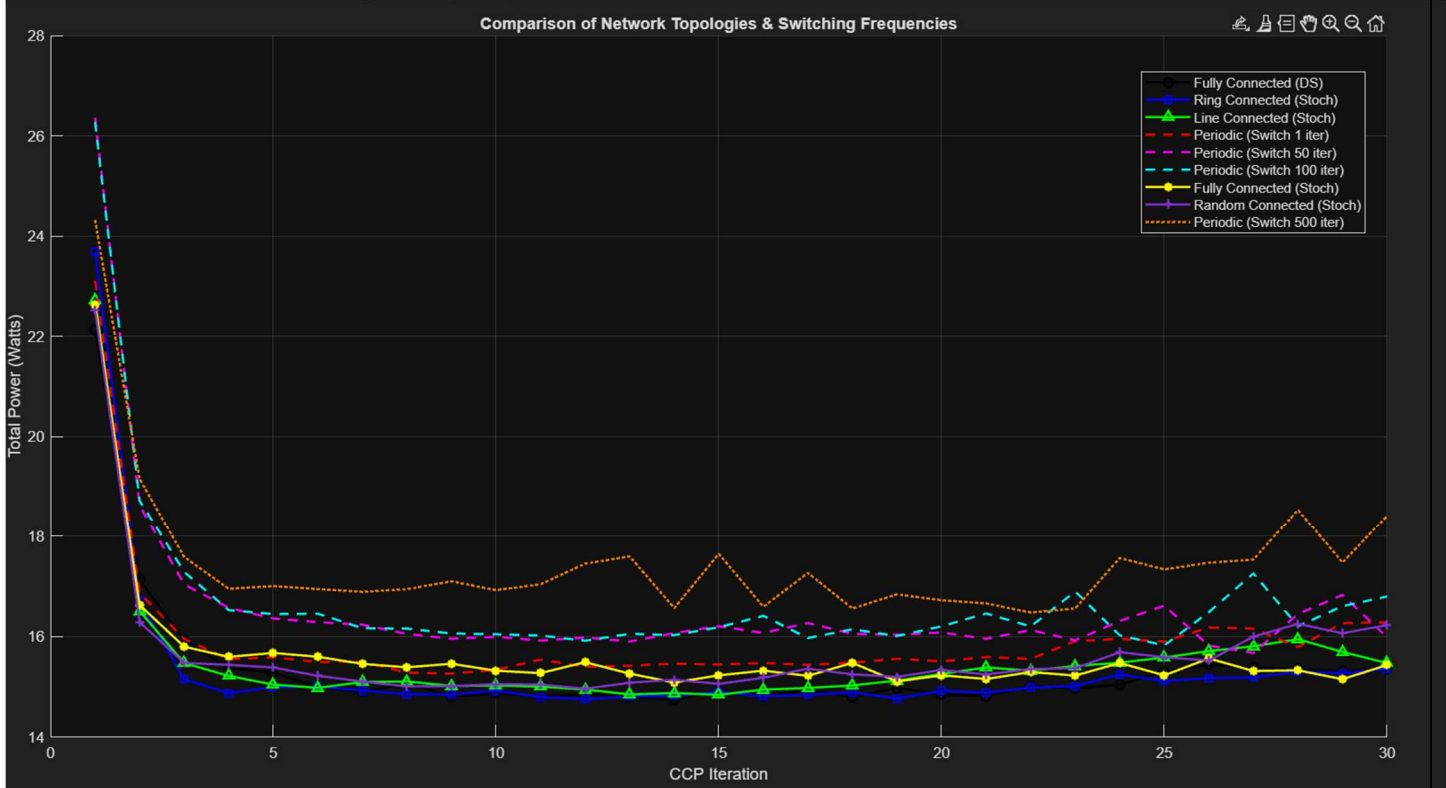
Scenario 7: Fully Connected (Stochastic / Asymmetric)

- **Graph:** Everyone is connected, but weights are unequal.
- **Topology:** Directed Complete Graph.

Scenario 8: Random Connected (Stochastic)

- **Graph:** Connectivity is determined by a random number generator.

- **Topology:** Unstructured / Ad-hoc.
- **Matrix:** Generated using rand(N,N) and normalized.



As we can see with periodic one's with making the switching slower the misadjustment becomes larger.(Consensus_Loops is considered 1 because the size of our problem.)

Noisy Links:

If nodes simply calculate the average of these noisy values, the result will constantly fluctuate (jitter) and never converge to a stable solution so we use damping factor:

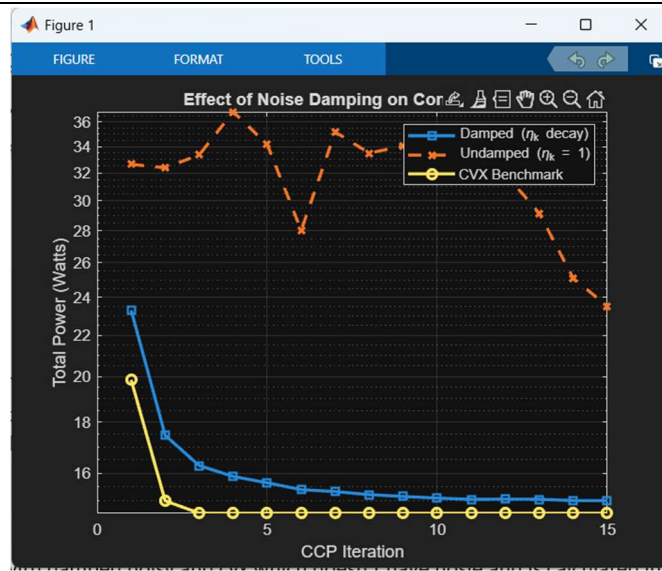
For $\alpha_k = \frac{1}{(k+1)^1}$, $\eta_k = \frac{1}{(k+1)^p}$ is appropriate with $\frac{1}{2} < p < 1$ so:

$$\eta_k = \frac{1}{(k+1)^{3/4}}.$$

The total idea is to calculate w base on weighted average of new received data and previous and because that with more iteration the previous and new data become more similar despite the beginning therefore our wheighting factor should decay eventually:

$$W_{new} = (1 - \eta_k)W_{old} + \eta_k(W_{average} + \text{Noise})$$

For noise total link_noise_var is considered to be 0.1 and it is complex normal with zero mean for comparison 3 scenario with damped noisy and cvx which doesn't have nosie and is calculated in different ccp loop so it's not effected by ccp update of noisy links. For connection graph is considered to be double stochastic.



Distributed Projected Gradient Descent:

We define the cost function as:

$$F_{total}(W) = \underbrace{\sum_{n=1}^N \sum_{m=1}^M |w_{n,m}|^2}_{\text{Objective}} + \rho \underbrace{\sum_{k=1}^K [\max(0, g_k(W))]^2}_{\text{Penalty for SINR Violation}}$$

For ρ we tune it so our SINR constraints are satisfied and projection on zero makes the SINR's not be like 15/12/2 where

Some of them are huge and some are small. Because each node is an antenna for projection it should project the beam on its own power constraint:

Take a step in the negative gradient direction to reduce power and fix SINR violations.

$$W_{temp} = V^{(i)} - \alpha_k \cdot d_i$$

C. The Projection (P_X):

Now, apply the Projection on Power Constraints. We strictly force the solution to stay inside the power limit "ball".

For each antenna row n :

- If $\|w_n\|^2 \leq P_n$, keep it as is.
- If $\|w_n\|^2 > P_n$, scale it down:

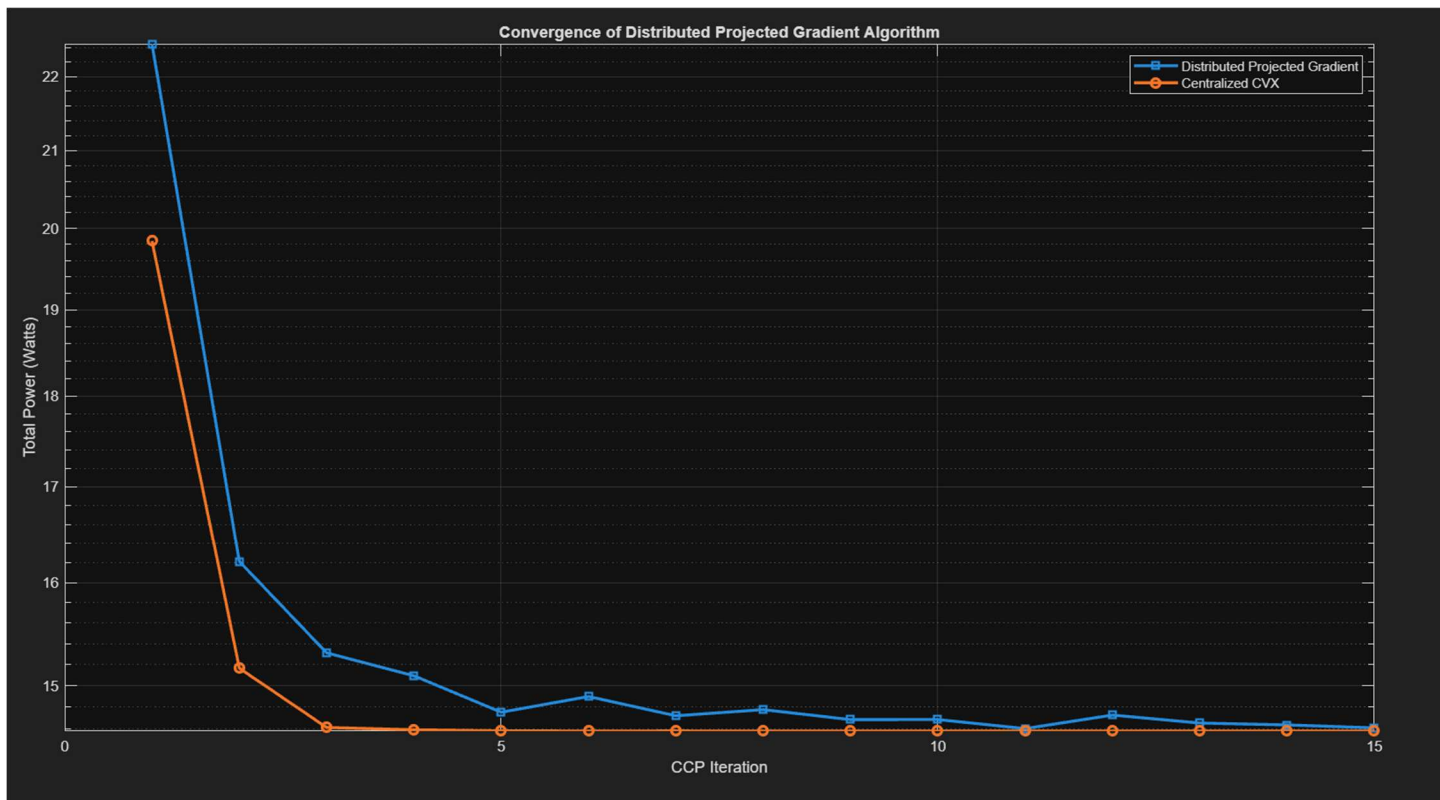
$$w_n = w_n \times \frac{\sqrt{P_n}}{\|w_n\|}$$

For this part also the graph is considered double stochastic.

Result:

With $\rho = 80$ and the same step size :

$$\alpha_k = 10/(1 + k) * ccp$$



```
=====
FINAL RESULT VERIFICATION
=====

--- Power Constraint Check ---
Antenna 1: Power = 1.8801 W, Limit = 10.0000 W [OK]
Antenna 2: Power = 3.5340 W, Limit = 10.0000 W [OK]
Antenna 3: Power = 6.3911 W, Limit = 10.0000 W [OK]
Antenna 4: Power = 2.8048 W, Limit = 10.0000 W [OK]

--- SINR Constraint Check ---
User 1: Target = 10.00, Achieved = 9.9978 [OK]
User 2: Target = 10.00, Achieved = 9.9950 [OK]
User 3: Target = 10.00, Achieved = 10.1267 [OK]

>> SUCCESS: All constraints satisfied.
>> |
```

Because of problem topology it's the same as Consensus-based subgradient methods for distributed sets.

Gossip:

Asynchronous Gossip-Based DPGD:

Asynchronous Node Selection

- Instead of a global clock where everyone wakes up at once, in each step, the algorithm randomly selects just **one pair of nodes** (Node A and Node B) to communicate.
- All other nodes in the network remain "asleep" and do not change their values. This mimics real-world ad-hoc networks where links are random and sporadic.

Action: The two selected nodes (A and B) exchange their current matrices and calculate the average:

$$W_{mix} = \frac{W_A + W_B}{2}$$

Both nodes A and B take this mixed value (W_{mix}) and try to improve it.

They calculate the gradient of a **Penalized Cost Function**.

$$W_{temp} = W_{mix} - \alpha \times \text{Gradient}$$

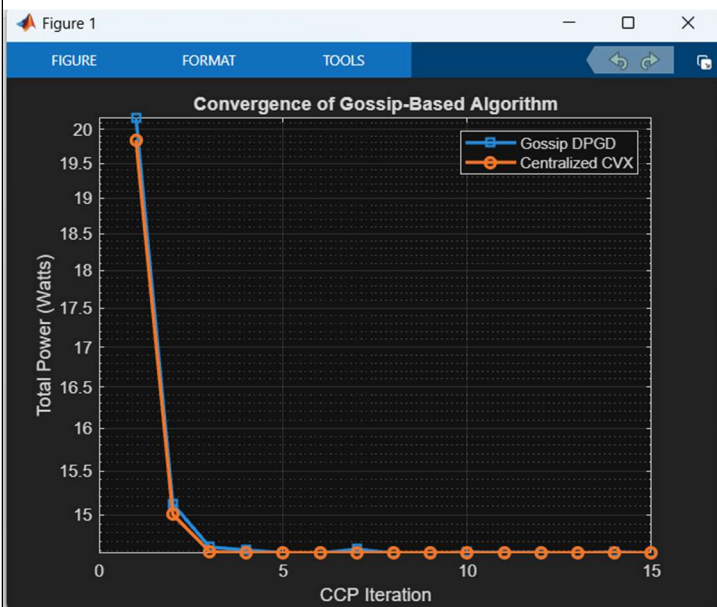
If the update pushed the power of an antenna beyond P_{max} , the node scales the relevant row of the matrix down so it sits exactly on the power limit boundary.

$$W_{new} = \text{Project}(W_{temp})$$

Step size is still:

$$\alpha_k = 10/(1 + k) * ccp$$

And because of this extension the graph doesn't have meaning anymore:



```
=====
FINAL RESULT VERIFICATION
=====

--- Power Constraint Check ---
Antenna 1: Power = 1.8441 W, Limit = 10.0000 W [OK]
Antenna 2: Power = 3.5330 W, Limit = 10.0000 W [OK]
Antenna 3: Power = 6.3786 W, Limit = 10.0000 W [OK]
Antenna 4: Power = 2.8237 W, Limit = 10.0000 W [OK]

--- SINR Constraint Check ---
User 1: Target = 10.00, Achieved = 9.9912 [OK]
User 2: Target = 10.00, Achieved = 9.9963 [OK]
User 3: Target = 10.00, Achieved = 10.0227 [OK]

>> SUCCESS: All constraints satisfied.
>>
```